

질의 처리율 가속화를 위한 멀티 CSD 기반 문서형 데이터베이스 설계

(Designing a Multi-CSD-based Document Database to Improve Query Throughput)

Hyungsun Yoo¹, Kihwan Kim¹, Junghyun Ryu¹, Hongsu Byun¹,
Myungcheol Lee², Jinchun Choi², Youngjae Kim¹
¹Sogang University and ²ETRI

Presenter: Kihwan Kim

2024. 12. 18.

Contents

- Background
- Motivation
- Design
- Evaluation
- Conclusion

Background

● CouchDB

- 허가형 블록체인 시스템(ex. 하이퍼레저 패브릭)에서 트랜잭션의 결과를 업데이트하는 State 데이터베이스로 사용됨
- 블록체인 상태(State)를 JSON 형태의 문서형으로 저장 및 관리함
- RESTful API를 지원하며, HTTP 프로토콜을 통해 데이터에 접근, 조작 가능함
- 클러스터 모드(Cluster mode)를 통한 확장성 있는 Cluster 시스템 구성이 가능함



Background

• CouchDB

- CouchDB는 문서형 기반 데이터베이스로서 사용자가 정의한 키와 값을 관리하므로 유연한 데이터 처리가 가능하며, 이러한 특성으로 복잡 질의 처리에 강점이 있음
- 다만, 저장된 모든 문서를 읽고 질의 처리를 하여 질의 수행 시간이 느리다는 문제가 존재함



```
{
  "_id": "5cf0029caff5056591b0ce7d",
  "firstname": "Jane",
  "lastname": "Wu",
  "address": {
    "street": "1 Circle Rd",
    "city": "Los Angeles",
    "state": "CA",
    "zip": "90404"
  },
  "hobbies": ["surfing", "coding"]
}
```

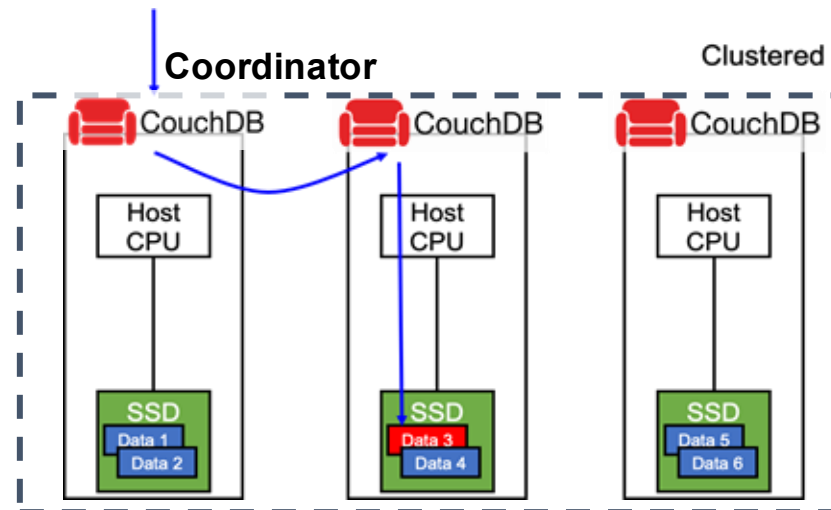
JSON 기반 data 예시

Background

• Cluster mode in CouchDB

Cluster 시스템 구축을 위한 CouchDB의 지원 기능

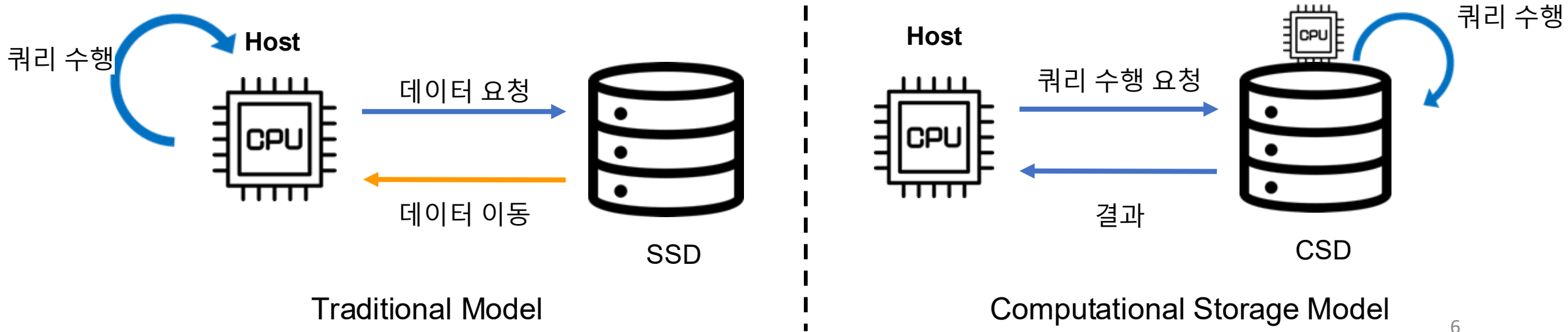
- Cluster mode는 샤딩(Sharding)과 복제(Replication)을 통해 균일한 데이터 분배와 노드 장애에 대처함
- 노드 추가를 통한 수평적 확대가 가능하며, 요청을 받은 노드는 Coordinator가 되어 I/O 라우팅을 수행함



Background

- **CSD** (Computational Storage Drive)

- In-Storage Processing(ISP) 기술을 사용하여, 다음과 같은 효과가 있음
 - 저장 장치로의 Task 오프로딩
 - 장치 간 데이터 이동량을 감소



Background

- **CSD** (Computational Storage Drive)
 - In-Storage Processing(ISP) 기술을 사용하여, 다음과 같은 효과가 있음
 - 저장 장치로의 Task 오프로딩
 - 장치 간 데이터 이동량을 감소
 - FPGA 기반의 가속기가 내장되어 특정 연산에 최적화된 성능을 보이는 **SmartSSD**^[1]와 자체적으로 ARM 기반 프로세서를 SSD 내부에 탑재하여, Linux 기반 운영체제에서 프로그래밍이 가능한 **Newport CSD**^[2]가 존재함

[1] Samsung Electronics, "SmartSSD," Apr. 6, 2022. [Online]. Available: <https://samsungsemiconductor-us.com/smartssd/>

[2] NGD Systems, "Newport CSD," Apr. 6, 2022. [Online]. Available: <https://www.ngdsystems.com/solutions#NewportSection>

Motivation

- Cluster 환경에서 문서형 데이터베이스인 CouchDB의 느린 성능을 극복하기 위해 노드를 추가하는 **성능 Scale-out 방식은 다음과 같은 한계가 존재함.**
 - 네트워크 지연: OS와 파일 시스템을 통해 네트워크 및 스토리지 하드웨어에 접근하기 때문에 다수의 레이어에서 지연 발생^[3]
 - 트래픽 처리로 인한 CPU 사용량 증가^[4]

→ 본 연구는 CSD를 활용하여 **단일 노드의 성능을 높이는 Scale-up 방식을 제안함.**

[3] M. Gabriel Haas and Viktor Leis, "Exploiting directly-attached nvme arrays in DBMS," 10th Conference on Innovative Data Systems Research, CIDR 2020, Amsterdam, The Netherlands, January 12-15, 2020.

[4] Qizhen Zhang, Philip A. Bernstein, Badrish Chandramouli, Jiasheng Hu and Yiming Zheng, "DDS: DPU-optimized Disaggregated Storage," Proceedings of the VLDB Endowment, vol. 17, no. 11, 2024.

Motivation

- CouchDB 질의 처리 가속화를 위한 CSD 활용 Constraints

Constraint 1) CSD에서의 연산 수행 명령을 위한 Host 단 **I/O Manager의 부재**

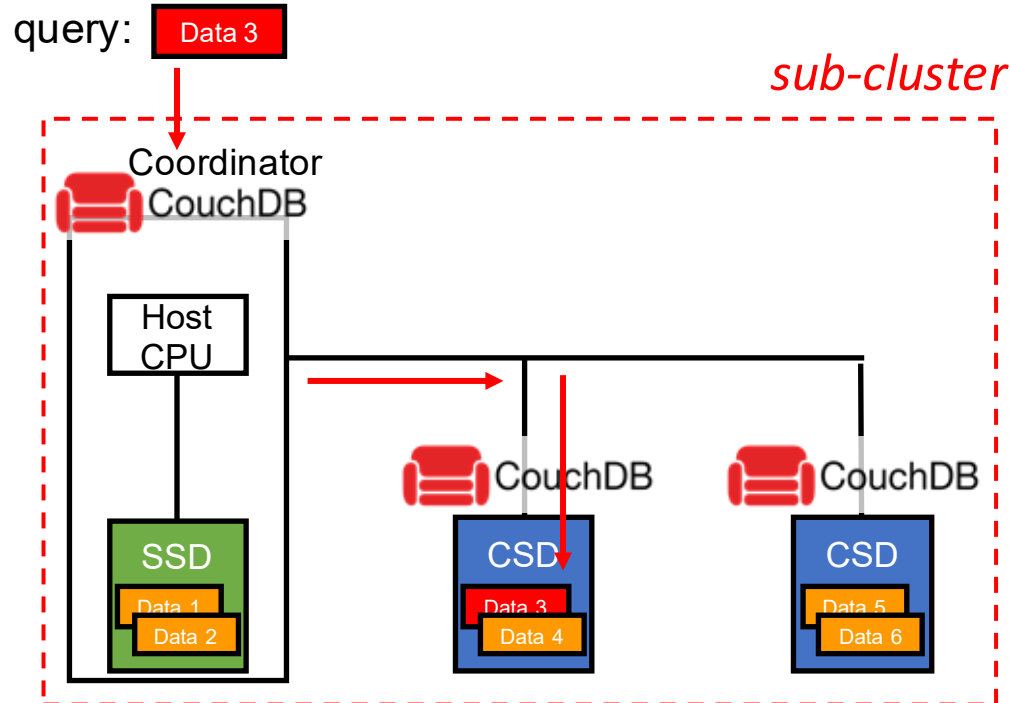
- CSD에서의 연산 요청 및 결과 취합의 역할을 수행하는 소프트웨어 스택이 없음

Constraint 2) Operation 오프로딩을 위한 **Data 배치 정책 설계의 복잡성**

- CSD는 In-data에 대해서 연산을 수행하며, 특정 Operation을 오프로딩한다면, CSD의 연산을 위한 적절한 Data 배치가 수행되어야하고, 이는 CouchDB의 데이터 처리 유연성을 저해함

Design

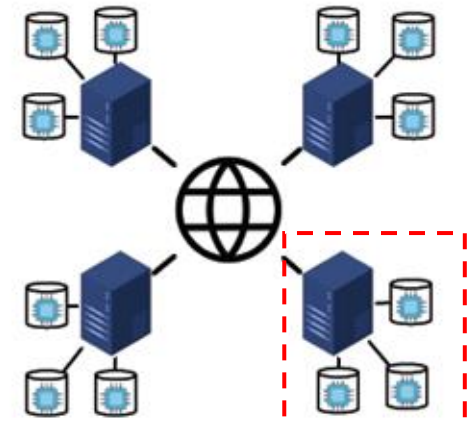
- OS가 탑재되어 CouchDB를 자체적으로 구동 가능한 **Newport CSD**를 단일 노드 내의 서브 노드로 등록하고, CouchDB의 **Cluster-mode**라는 기능을 활용하여 단일 노드 내 **sub-cluster** 시스템을 제안함.



Cluster mode using CSDs



(a) 전통적인 분산 DB

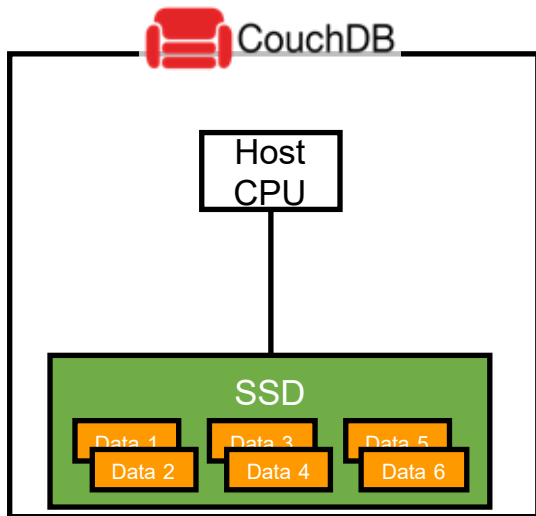


(b) CSD 활용 클러스터 DB

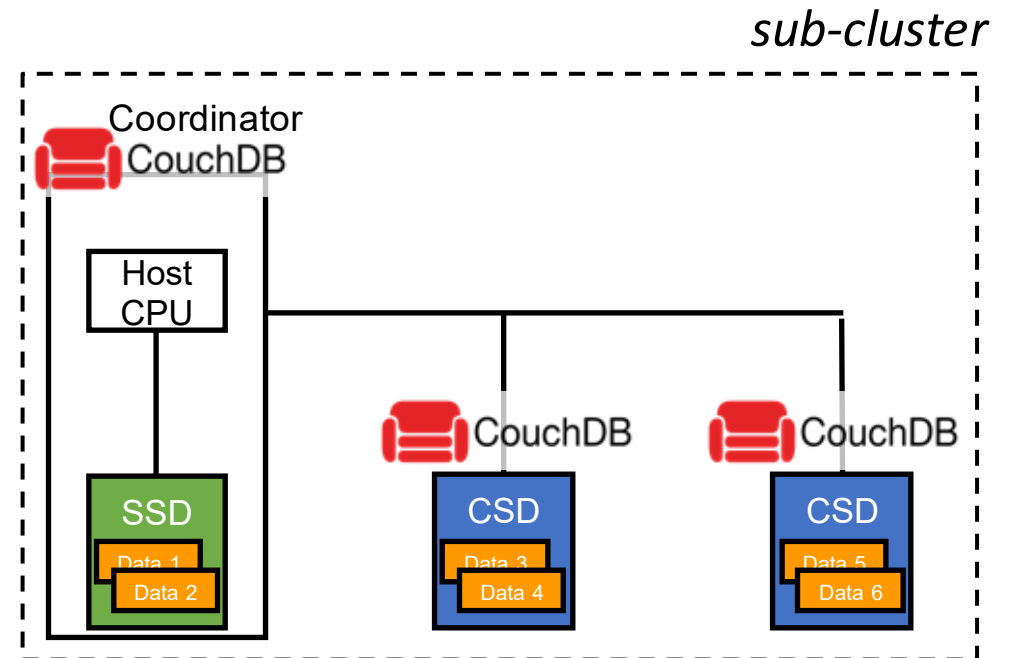
sub-cluster

Design

- Cluster-mode는 기존의 두 가지 Challenge을 해결할 수 있음
 - (1) Cluster-mode가 **I/O Manager**의 역할을 수행함
 - (2) Operation offloading이 아닌, 하나의 operation을 분할된 노드에서 수행함



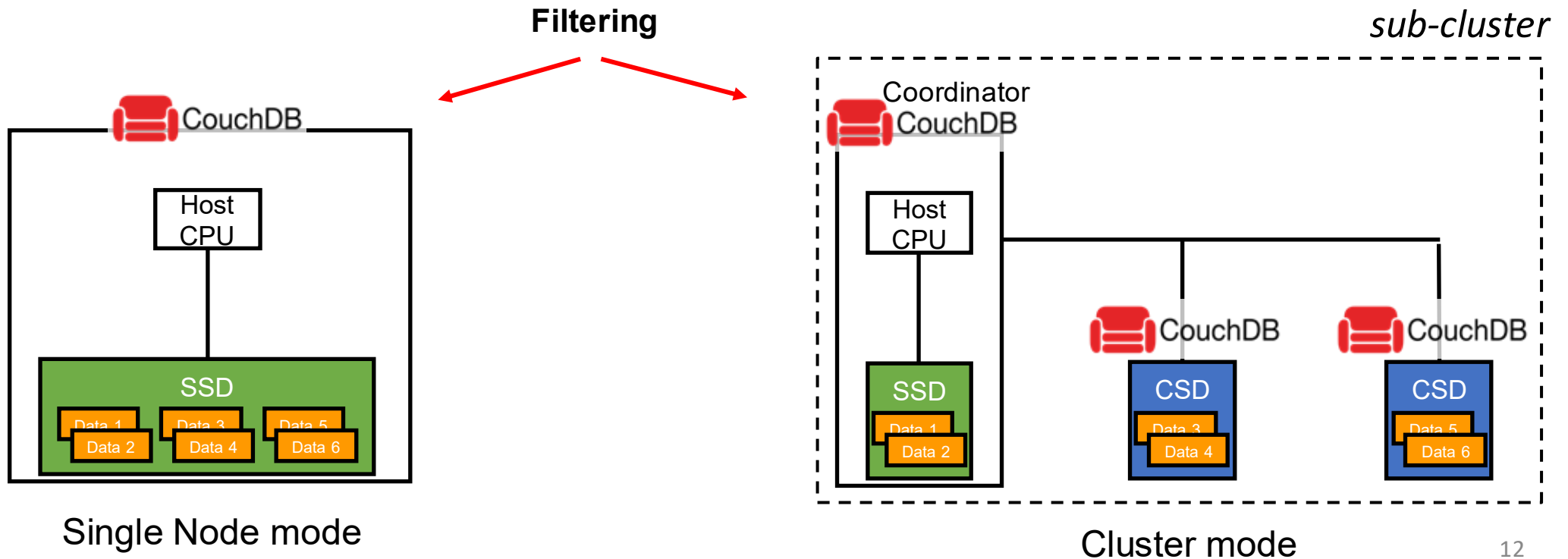
Single Node mode



Cluster mode

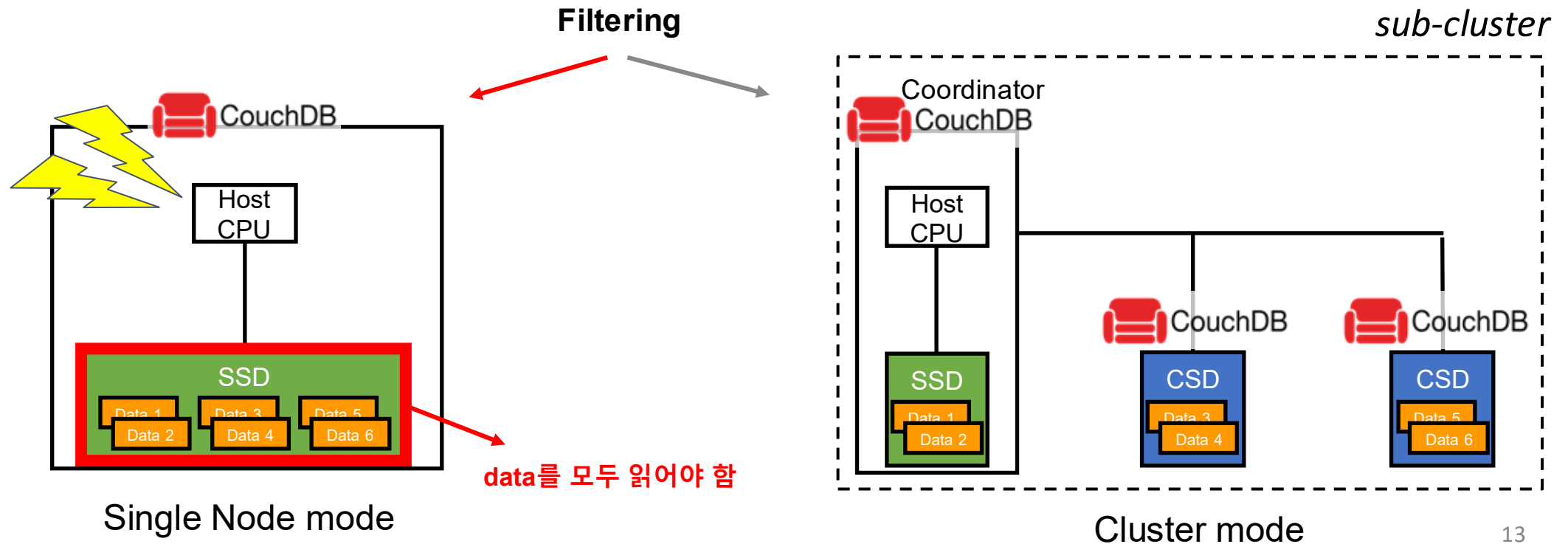
Design

- Cluster-mode는 기존의 두 가지 Challenge을 해결할 수 있음
 - (1) Cluster-mode가 **I/O Manager**의 역할을 수행함
 - (2) Operation offloading이 아닌, 하나의 operation을 분할된 노드에서 수행함



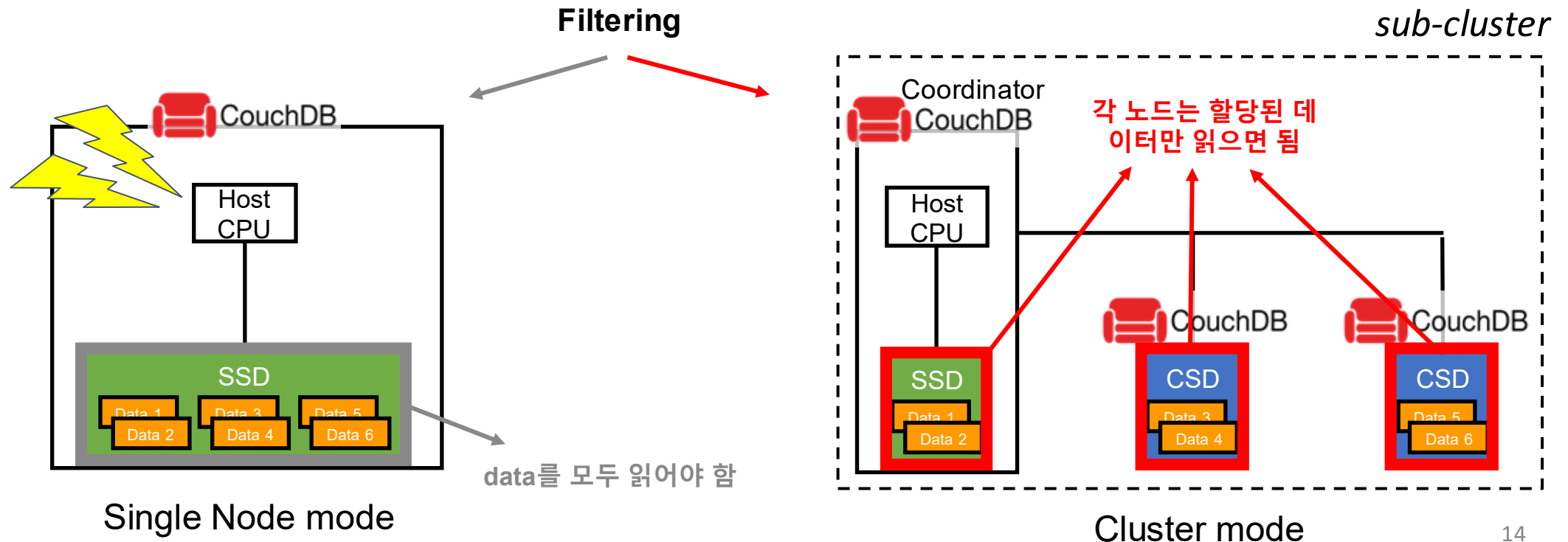
Design

- Cluster-mode는 기존의 두 가지 Challenge을 해결할 수 있음
 - (1) Cluster-mode가 **I/O Manager**의 역할을 수행함
 - (2) Operation offloading이 아닌, 하나의 operation을 분할된 노드에서 수행함



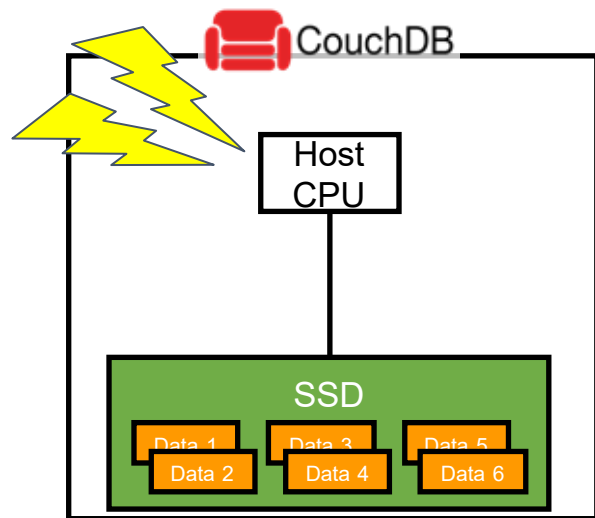
Design

- Cluster-mode는 기존의 두 가지 Challenge을 해결할 수 있음
 - (1) Cluster-mode가 **I/O Manager**의 역할을 수행함
 - (2) Operation offloading이 아닌, 하나의 operation을 분할된 노드에서 수행함



Design

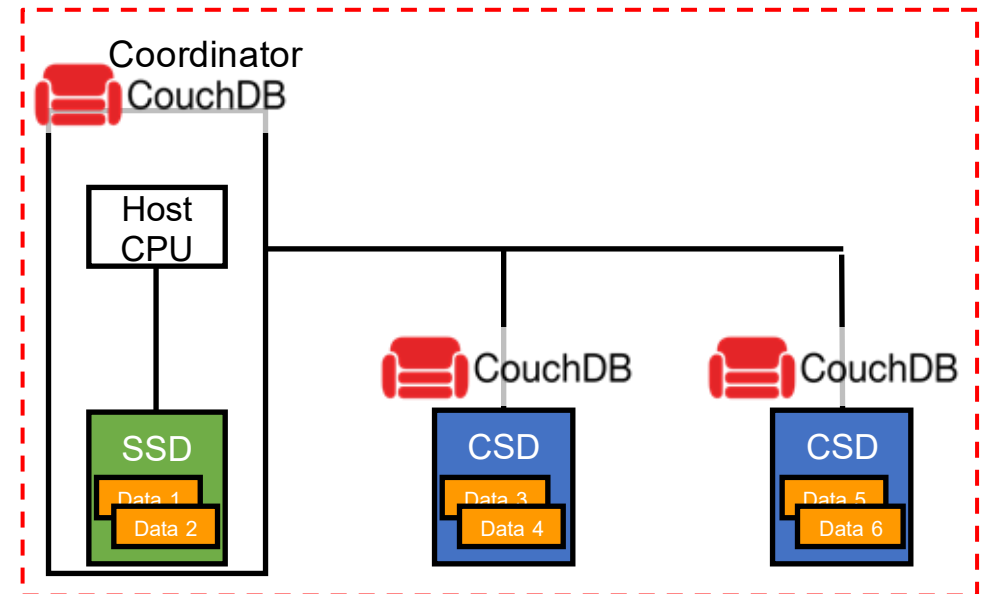
- Cluster-mode는 기존의 두 가지 Challenge을 해결할 수 있음
 - (1) Cluster-mode가 **I/O Manager**의 역할을 수행함
 - (2) Operation offloading이 아닌, 하나의 operation을 분할된 노드에서 수행함



Single Node mode



각 노드가 관리하는
데이터 수 경감 효과



Cluster mode

Evaluation Setup

- 아래와 같은 실험 환경에서 성능을 측정함.
 - Host System : 호스트 서버만 사용한 단일 CouchDB 실행
 - CSD System : 호스트 서버와 CSD 장치를 모두 사용한 클러스터 모드의 CouchDB 실행
 - 128KB크기의 문서를 100,000개 가지고 있는 데이터 베이스에 Filtering 질의 수행

표 1: 호스트 서버 상세 설명

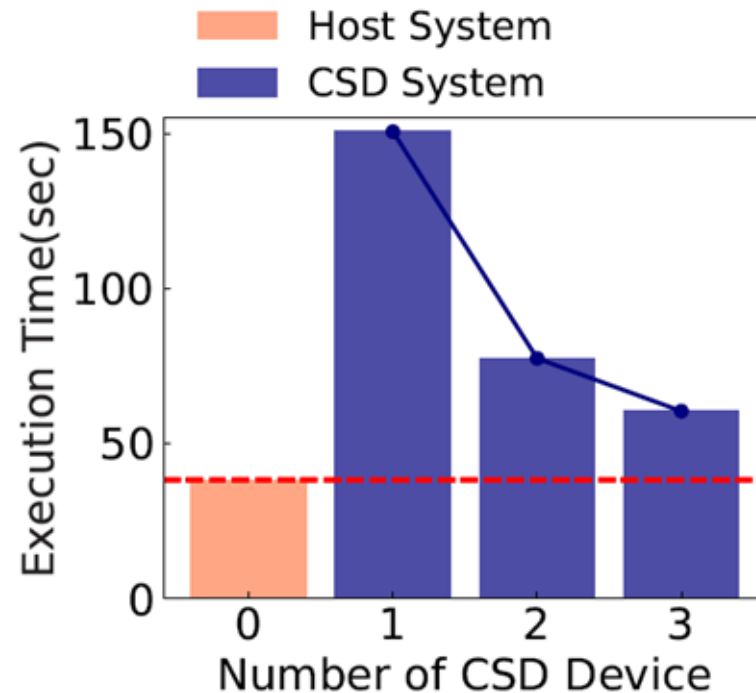
CPU	AMD EPYC™ 7352, 48 Cores (96 Threads), 2.3GHz (Up to 3.2GHz)
Socket	2 NUMA Node
Memory	256 GB (64 GB × 4) DRAM (DDR4)
OS	Centos 7.92.2009 (Core) / Linux Kernel 4.14
Storage	Dell 1.92 TB SSD SAS 24Gbps ISE RI 512e

표 2: Newport CSD 상세 설명

Storage Capacity	8 TB	
Host Interface	Single Port PCIe Gen3x4 (U.2)	
NAND Flash Memory	Toshiba Flash Memory	
In-Storage Processing Engine	ARM Cortex-A53, 1.0 GHz, 4 Cores	
	Memory	8 GB DRAM (DDR4)
	OS	Linux Kernel 4.14

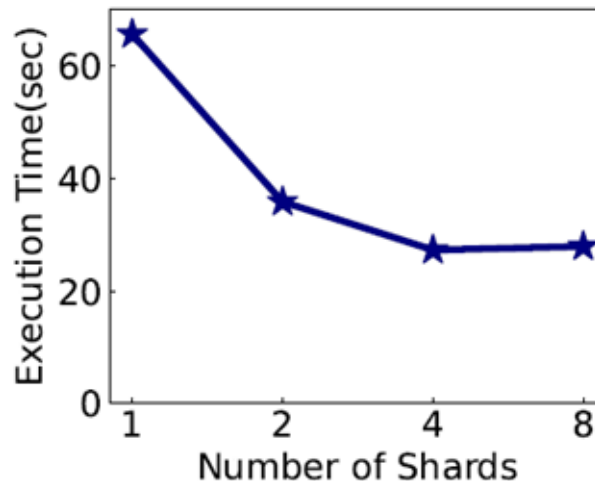
Evaluation

- CSD 장치를 추가했음에도 불구하고, 오히려 **쿼리 수행 성능이 악화**되어 **BEP(Break-Even Point)**조차 찾지 못하는 문제 발생



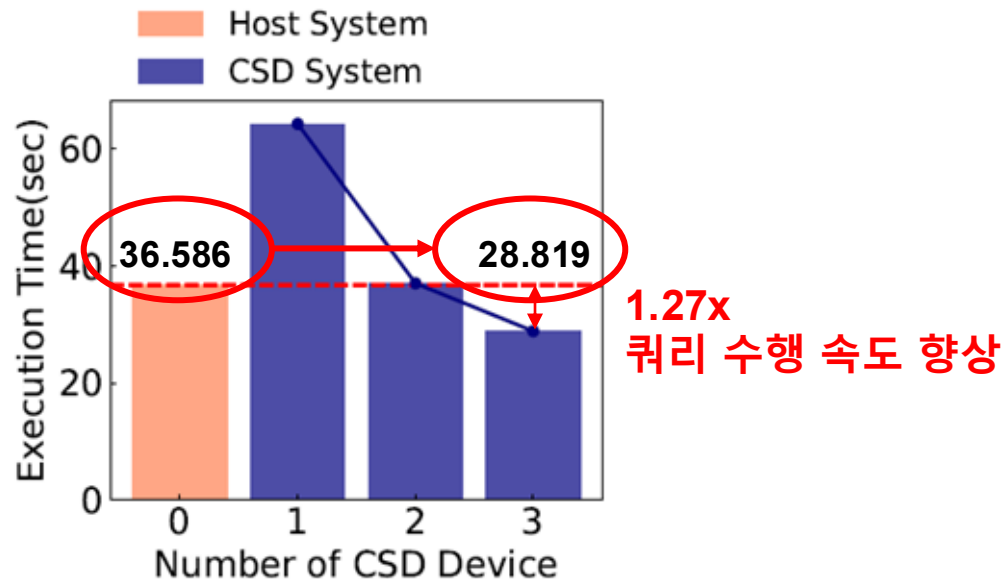
Evaluation

- CouchDB는 내부적으로 데이터베이스를 샤드(Shard)단위로 분할하여 병렬 처리함
→ 각 노드의 CPU 코어 개수에 따라, 샤드의 개수를 적절히 설정해주어야함
- 25,000개의 문서의 데이터베이스를 관리하는 단일 CSD 장치에서 쿼리 수행 시간을 측정한 결과, **샤드 개수가 4개로 증가함에 따라 성능 향상이 나타남**
 - 다만, 노드 내 병렬성 한계로 4개를 넘어설 때부터는 성능 향상이 나타나지 않음



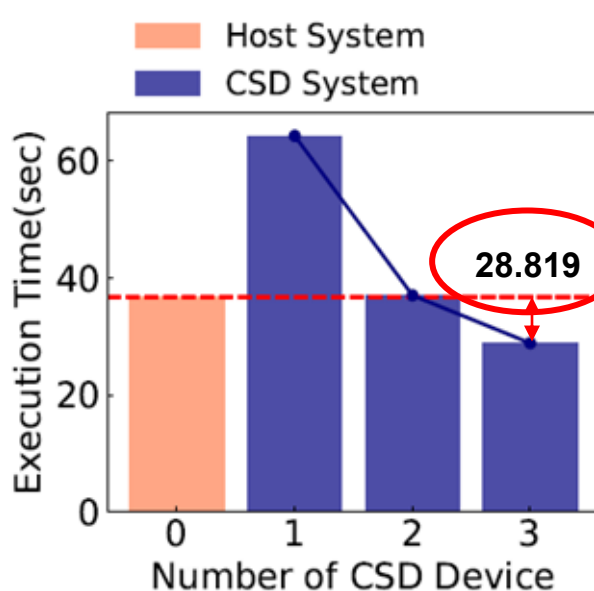
Evaluation

- 샤드 개수를 4개로 고정하고 같은 환경에서 실험한 결과, **CSD 개수가 3개일 때 성능 향상**이 이루어지는 지점을 발견함
 - Host 시스템에 비해서 약 **1.27배**의 성능 향상을 보임

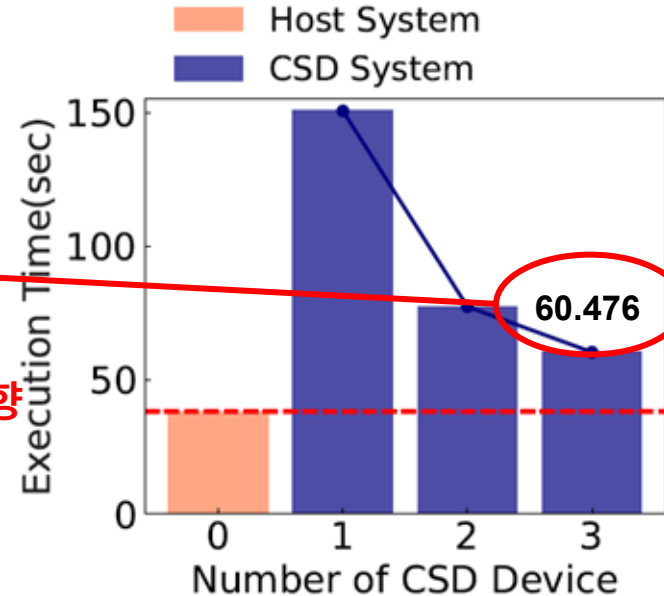


Evaluation

- 샤드 개수를 4개로 고정하고 같은 환경에서 실험한 결과, **CSD 개수가 3개일 때 성능 향상**이 이루어지는 지점을 발견함
 - Host 시스템에 비해서 약 1.27배의 성능 향상을 보임
 - 샤딩 적용 전 **CSD 시스템**에 비해 약 **2.09배**의 성능 향상을 보였음

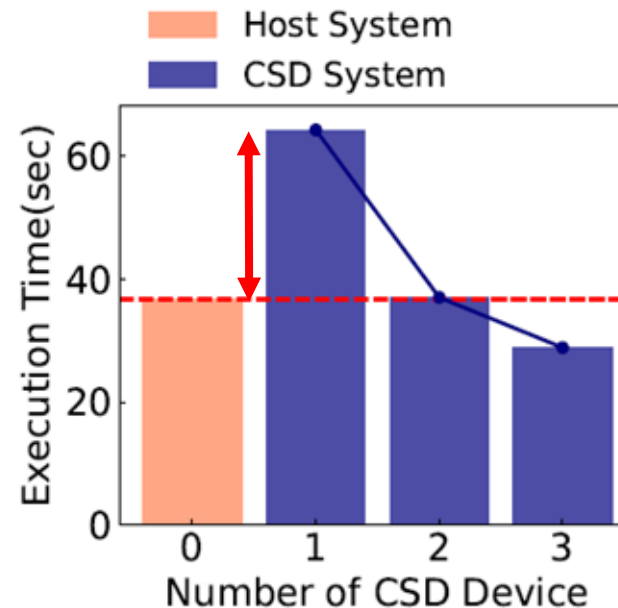


2.09x
쿼리 수행 속도 향상



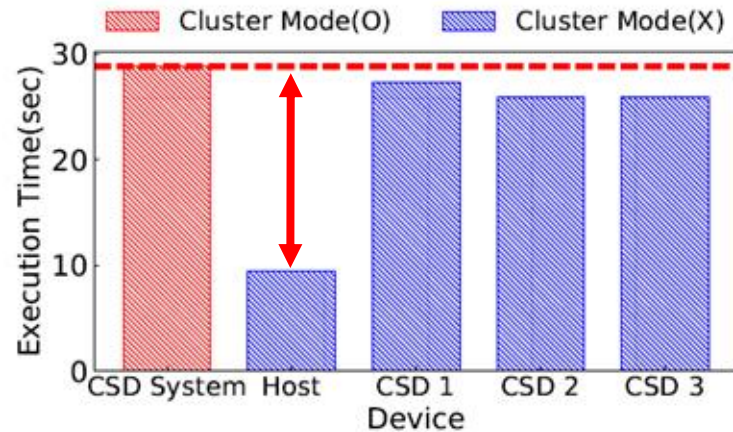
Evaluation

- 그러나 추가적인 장치를 활용했음에도, 여전히 CSD 개수가 1,2개 일 때는 성능 향상이 미미하거나 심지어 Host 시스템이 더 우수한 경우가 존재함
- 이는 클러스터 모드에서 장치별 **Computing power**를 고려하지 않은 데이터의 균등 분배가 이루어졌기 때문

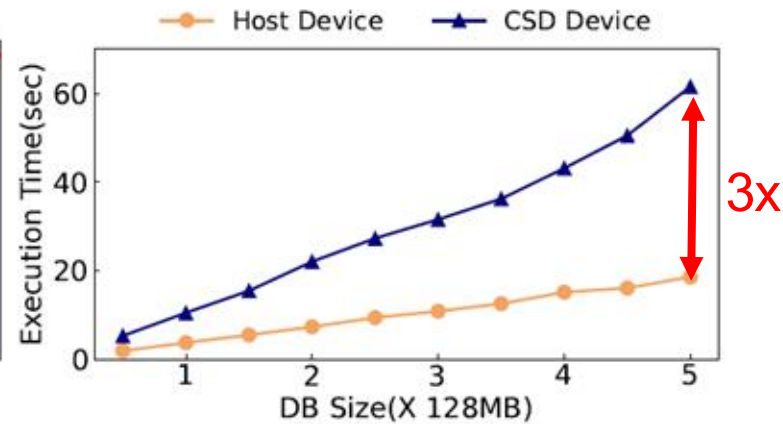


Evaluation

- 그림 (a)에서 볼 수 있는 것처럼 **CSD 시스템의 총 질의 처리 시간은 sub-cluster 시스템 내 가장 느린 장치(CSD 장치)에 의해 결정되는 것을 확인함**
- 그림(b)는 데이터베이스 크기를 증가시켜가며 장치별 쿼리 수행 시간을 측정한 결과이며, 각 지점에서 디바이스의 수행 시간을 비교해보면 **약 3배 정도의 성능 차이**가 나는 것을 확인함



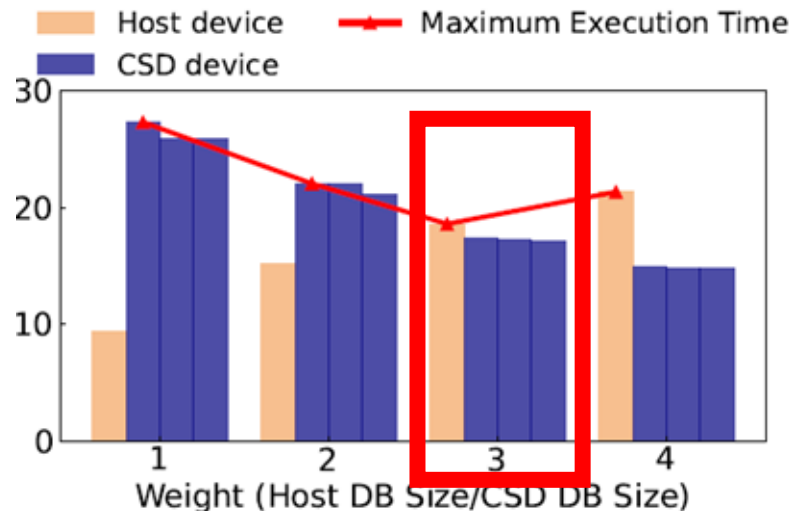
(a)



(b)

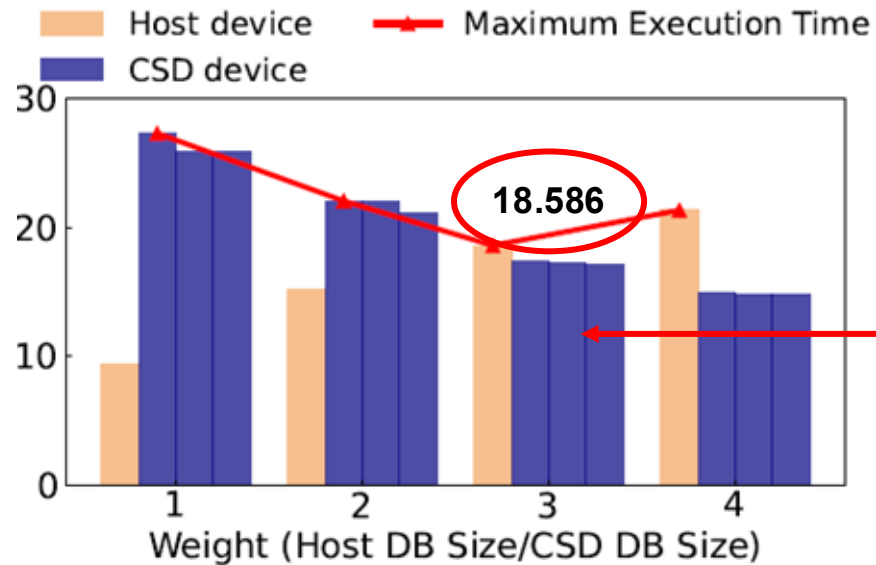
Evaluation

- 부하 분산을 적용한 CSD 시스템 성능 예측을 위해 임의로 각 노드에 DB 크기를 조절하여 배치함
 - (Recap) CSD 시스템의 쿼리 수행 시간은 가장 느린 장치의 수행 시간에 의해 결정됨
- **Computing power의 차이를 고려하여 가중치를 3으로 뺐을 때 가장 빠른 질의 처리 속도를 보여줄 것으로 확인됨**

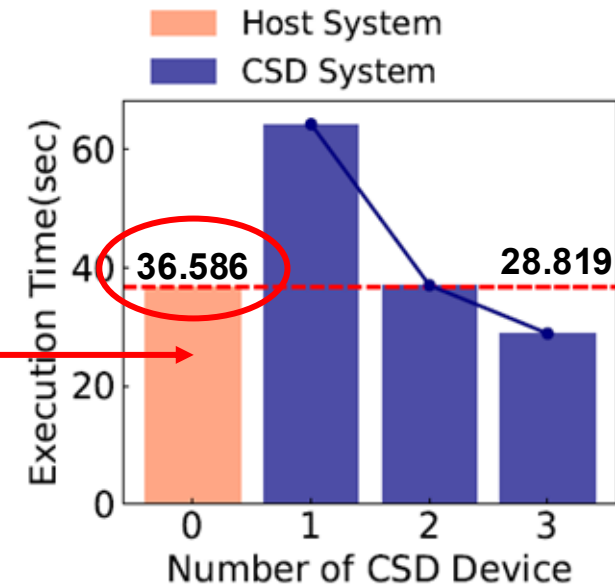


Evaluation

- 기존 시스템들과 성능을 비교했을 때,
 - **Host 시스템 대비 약 1.97배** 성능 향상이 예측됨



데이터 분산 배치 적용 o

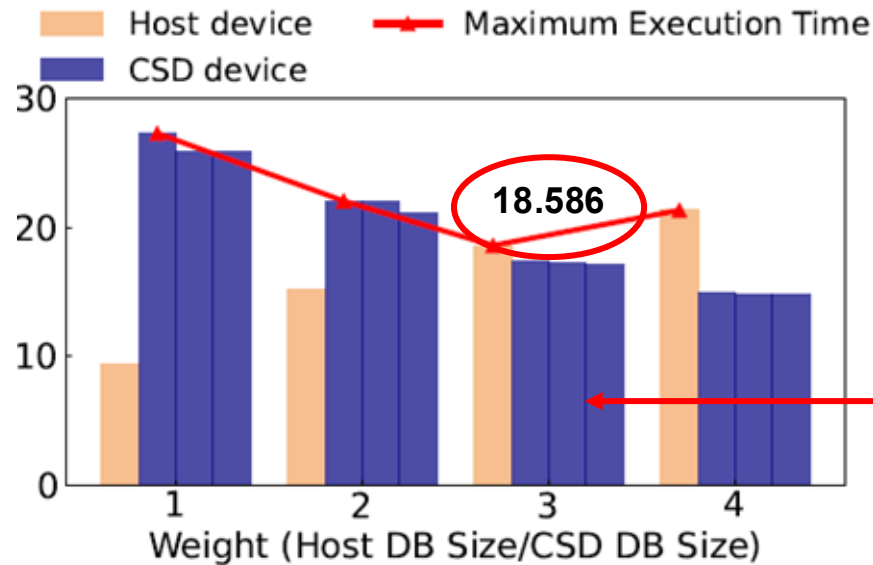


데이터 분산 배치 적용 x

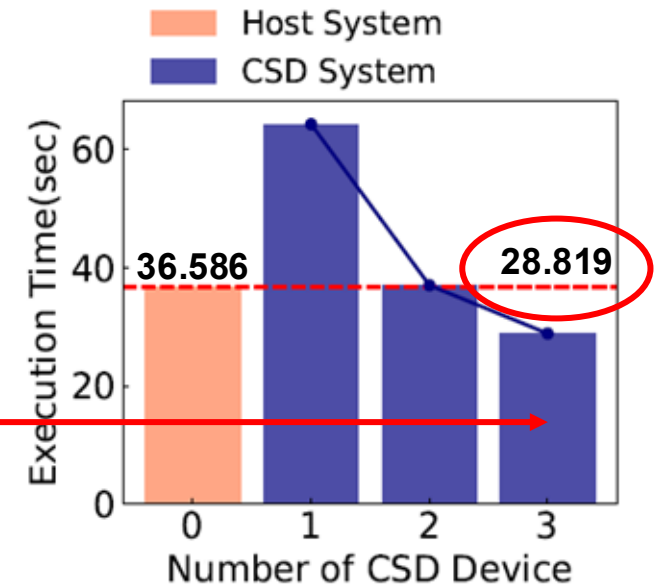
1.97x

Evaluation

- 기존 시스템들과 성능을 비교했을 때,
 - 샤딩이 적용되었지만 부하 분산이 적용되지 않은 기존 CSD 시스템 대비 약 1.55배 성능 향상이 예측됨



데이터 분산 배치 적용 o

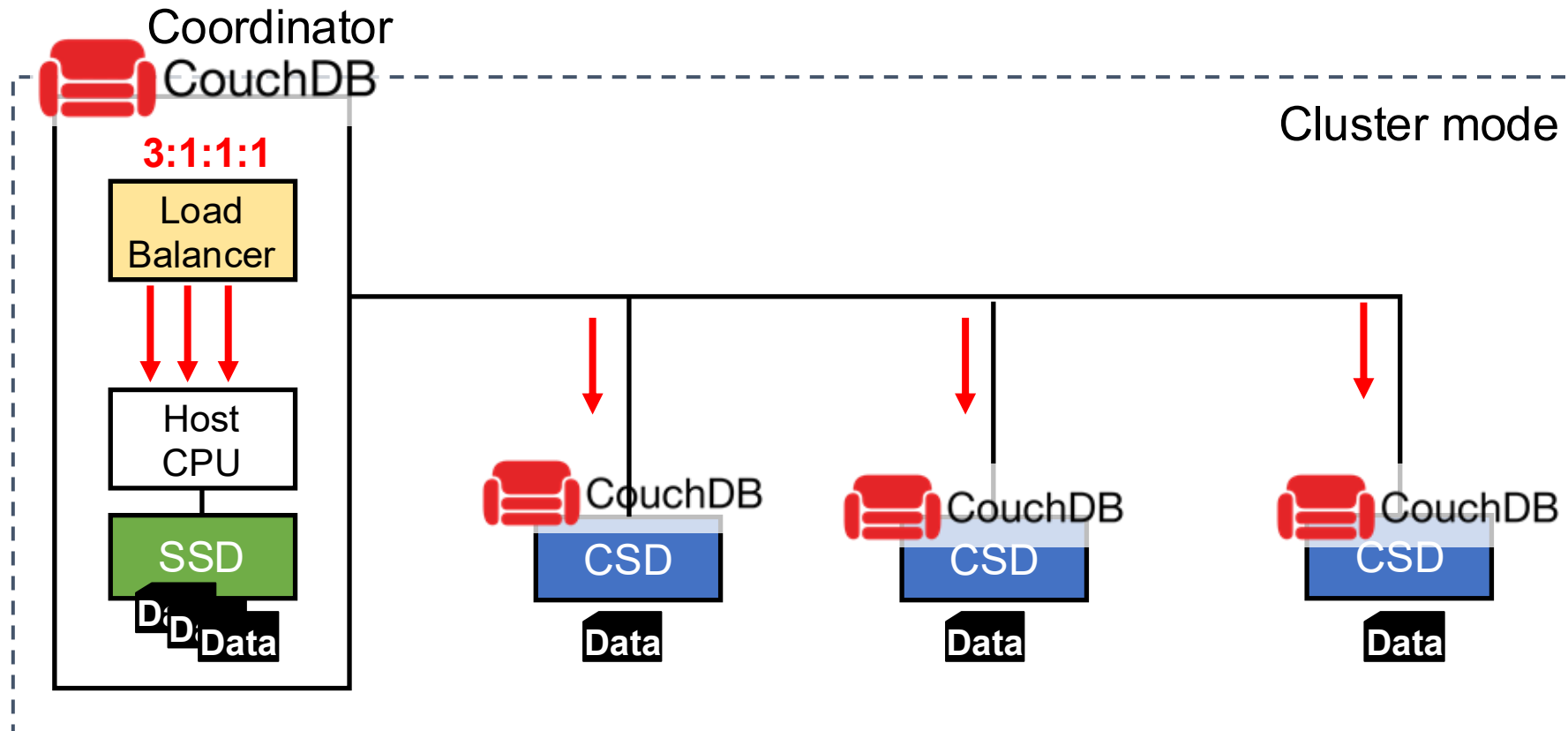


데이터 분산 배치 적용 x

1.55x

Advanced Design

- 따라서 특정 노드가 더 많은 데이터를 관리할 수 있게 **Computing power**를 고려한 데이터 분산 배치가 필요함



Conclusion

- Cluster 시스템의 질의 처리 성능 확장을 위해 Scale-out의 방식이 아닌 단일 노드내 성능 향상을 위해 **CSD를 활용한 Scale-up** 방식을 제안
- CSD를 활용하기 위한 **I/O 스택의 대체, 활용 방식의 방향성** 제시
 - CouchDB의 Cluster mode를 이용한 I/O Manager 역할 수행
 - 노드 내 샤드 개수 최적화를 통한 병렬 처리 활용
- 서브 클러스터 시스템 구축을 위하여, **장치 내 병렬성과 장치 간 연산 속도 차이**를 고려한 시스템 설계가 필요함



서강대학교
SOGANG UNIVERSITY

Thank You

서강대학교
SOGANG UNIVERSITY